当我把 AI 变成一个"算法"：Skill 工程化设计的心路历程

原创 腾讯程序员 腾讯技术工程 May 13, 2026, 2:24 AM 广东 319 listened



作者：peihanyu

引言

> 你有没有这样的经历——给 AI 写了一大堆规则，越写越长，它反而越来越不听话？这篇文章讲的是：我如何从"写提示词"走向"造执行环境"，让 Agent 从一个不可控的对话机器人，变成一个精确、可恢复、可审计的工程化组件。

我想做一件什么事

先说结论。

我的目标是：**把 Agent 当成一个算法来用。**

什么叫"当算法用"？就是——你给它输入，它给你指定格式的输出。中间的推理过程你不关心，但结果是确定的、可预期的。

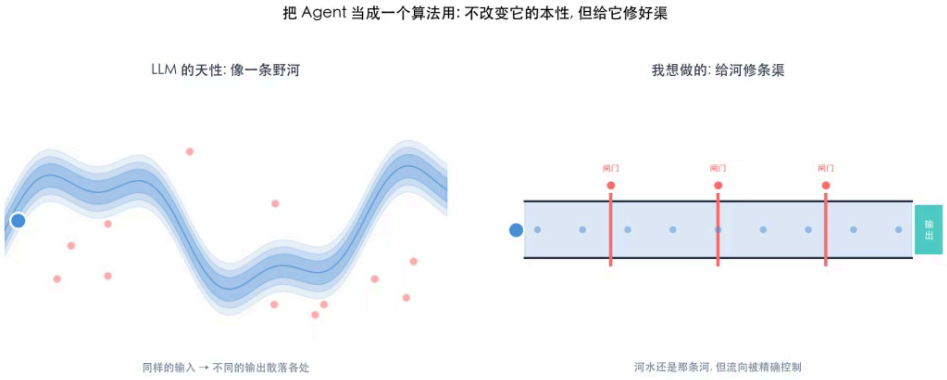
听起来很简单？但 LLM 天生不是这样的东西。它是概率模型，不是函数。每次调用它都像是在概率空间里掷骰子——你能控制骰子大致落在哪一片区域，但不能控制精确的落点。

这就引出了两个真实的工程痛点：

痛点 1. Token 是钱，试错是浪费。 Agent 在模糊需求前反复揣摩、多轮尝试、走了一半发现方向不对再重来——每一步都在烧 Token。一个"创建权限集"的操作如果要 Agent 自己摸索流程，可能跑 5 轮对话才能做对。但如果有人替它把路铺好，1 轮就够了。

痛点 2. 路径不确定，每次执行像开盲盒。 同一个提示词，换个模型、换个上下文顺序，输出可能完全不同。用户今天走通了，明天换了个说法就翻车。这种不确定性在 demo 里可以容忍，但在生产环境里是灾难。

我想做的是：不改变河的本性，但给它修好渠。



当它出错的时候，我们不去责怪它模型能力不够强，而是去思考执行环境还缺少了什么关键因素。让 Agent 保留它最擅长的"理解人话、做判断、组织表达"的能力，但把所有确定性的事——流程顺序、数据格式、API 调用、状态管理——全部从它脑子里拿出来，交给一个它触碰不到的确定性程序。

这样做之后，Agent 的角色就变了：它不再是一个"什么都要自己想"的自由意志体，而是一个嵌入在精密轨道里的决策引擎——输入是用户的自然语言，输出是结构化的 JSON 参数。中间的推理是它的事，但推理之外的一切，都被轨道接管了。

接下来我会从大到小，一个模块一个模块拆开这套系统。

出发点：当规则写到 200 行，AI 开始叛逆

最初我也是按主流做法来——把所有规则写进一份 SKILL.md：

> "你是某业务平台助手。1. 使用 CLI 操作。2. 先问用户要参数。3. 执行前确认。4. 注意某字段必须先做白名单……（200 行规则）"

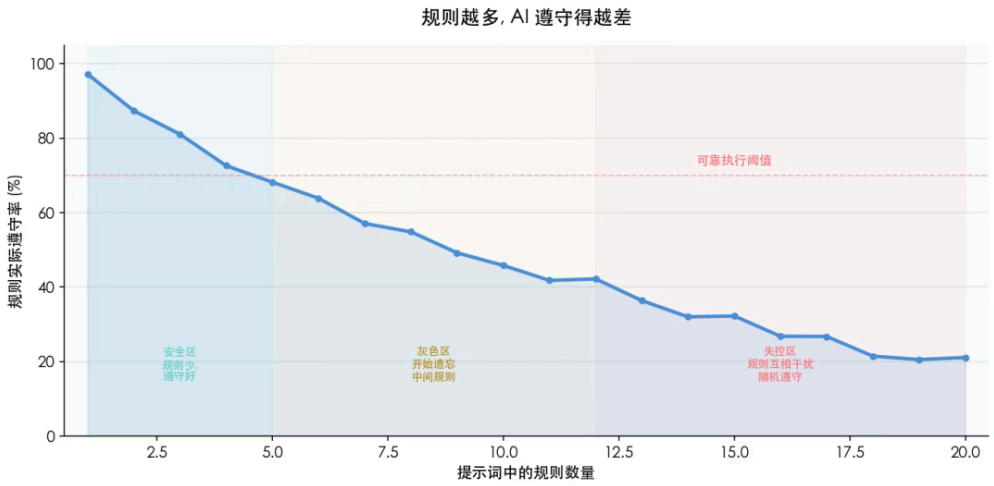
跑了几轮就发现：**规则越多，AI 遵守得越差。**

它会跳步骤、把字段名串到别的接口上、不问就执行。我一开始以为是模型不够聪明——换了几个模型都一样。这说明问题不在模型，在结构。

LLM 的注意力不是你以为的那样

很多人有个误解："128K 上下文 = AI 能同时注意 128K 信息"。实际上，LLM 的注意力更像人类读一本很厚的操作手册——你知道所有内容都在里面，但真正能"活跃"在大脑中的只有你刚读过的那几页。

把 200 条规则一次性灌进去，效果就像让一个实习生第一天上班时，你把公司全部规章制度从头到尾读一遍然后说"好了你去干活吧"。他能记住什么？大概就是最开头的几条和最后强调的那条。



真正该问的问题

问题不是"怎么写更好的提示词"。

而是：**怎么设计一个让 AI 在每个时刻都只需要关注最少信息的执行环境？**

这个问题一旦问出来，后面所有的设计就像多米诺骨牌一样自然倒下了。

第一根支柱：CLI 接管一切确定性的事

AI 到底在哪里翻车？

复盘那些出错的 case，规律很清晰：

- 拼 HTTP 请求时漏了 header ×
- 写 YAML 时缩进错了 ×
- 调接口时把 `userId` 写成了 `user_id` ×
- 理解用户说的"帮我创建一个项目" ✓
- 从三个选项里选出最合适的 ✓
- 用友好的语气回复"项目已创建" ✓

凡是涉及精确格式、固定流程的事，AI 不靠谱；凡是涉及理解、判断、表达的事，AI 很在行。 这就像一个绝顶聪明的战略家让他去填税务表格——他照样漏格子。不是笨，是能力类型不匹配。

决策：引入 CLI，让 Agent 只做"大脑"

我在 Skill 里引入一个 **CLI 程序**——一个 bash 脚本——作为 Agent 的执行层。

角色	Agent（大脑）	CLI（手脚）
做什么	理解意图、收集参数、组织回复	调 API、写文件、管状态
沟通方式	只输出 JSON 参数	只返回 JSON 结果

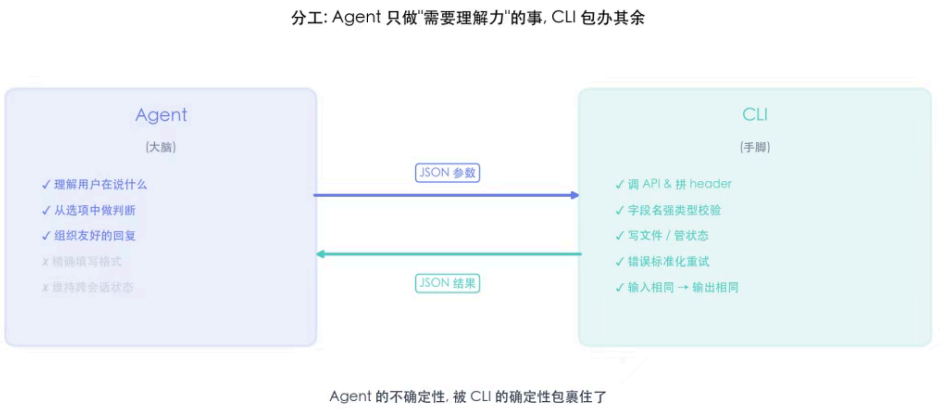
角色	Agent（大脑）	CLI（手脚）
不确定性	有（被限定在决策范围内）	无（相同输入 = 相同输出）

"创建一个项目"这个动作——

旧方式：Agent 自己拼 HTTP 请求 → 漏 header / 字段名拼错 / 认证 token 格式不对

新方式：Agent 只说 `"调用 create_project，参数：name=foo，host=bar.com"` → CLI 负责剩下一切

Agent 的不确定性，被 CLI 的确定性包裹住了。



Agent 的不确定性, 被 CLI 的确定性包裹住了

但 CLI 在这套系统里的角色，远不止"执行"

大部分人引入 CLI 就是为了"稳定执行"。这是对的，但如果只做到这一步，你解决的只是单次操作的可靠性。而我的 CLI 做了一件更深的事：**它接管了 Agent 的上下文管理。**

这里要先说一下大部分 Skill 方案面临的一个工程困境——

工具多了之后的死亡螺旋

当你的 Skill 背后有 5 个工具时，把每个工具的参数说明写在 SKILL.md 里没问题。但如果有 20 个？50 个？

常见的解决思路和它们的局限：

方案	做法	局限
全量写进 SKILL.md	所有工具 schema 平铺在提示词里	上下文爆炸，后面的工具 Agent 记不住
拆成多个 reference 文件	每个工具一个 md，Agent 按需 read_file	Agent 得先判断该读哪个文件，又是一层不确定性
用 MCP tool schema	依赖 IDE 的工具定义协议	工具数量多时 schema 占用大量 token，且不可定制
写更好的索引	做一份工具目录让 Agent 查阅	索引和实际 schema 容易不同步，维护成本高

这些方案的共同问题是：**Agent 仍然需要做"我该读什么"的决策**。它要调用 `read_file` 工具去读参数文档，而这个调用本身就可能出错——读错文件、读了过时的缓存、读出来太长又占了上下文。

我的做法：CLI 强制接管，Agent 不读文件

在这套系统里，Agent **永远不需要调用 read_file 去读工具说明**。它的信息获取路径被 CLI 完全托管：

```
Agent 被激活
    ↓
CLI discover → 自动同步后端工具列表 → 生成精简索引
    ↓
Agent 读 tools-index.md（一份表格，name + 一句话描述）
    ↓
用户说"帮我创建项目"
    ↓
Agent 调 CLI: create_project --meta-json --required-only
    ↓
CLI 返回该工具的必填字段 JSON（已过滤系统字段、已附加规则）
    ↓
Agent 根据 JSON 向用户收集参数 → 调 CLI 执行
```

这条链路上，Agent 从未"主动读取"过任何文件。**CLI 就像一个只给 Agent 看它该看的信息的看门人——什么时候给什么，由 CLI 决定，不由 Agent 决定。**

这解决了一个很微妙的问题：当 Agent 有"主动读文件"的能力时，它可能读错、多读、漏读。但当这个能力被收走之后，**它犯错的可能性被结构性地消灭了**——因为所有信息都是 CLI"喂"给它的，而 CLI 是确定性的程序。

CLI 工具管理的三层分离

为了让"50 个工具也不撑爆 Agent"，CLI 对工具信息做了三层分离：

层级	内容	Agent 何时可见	存储
索引层	工具名 + 一句话描述	始终可见	`tools-index.md`
元数据层	字段名、类型、是否必填、枚举值	用户请求某工具时按需加载	`tools/<name>.meta`
规则层	IGNORE/NOTE/ENUM 注解	融合在元数据输出中，Agent 不直接看到	`tools/<name>.rules`

这就像一个图书馆——你走进去先看到的是书架分类标签（索引层），不是每本书的全文。你选了一本书走过去，才看到它的目录（元数据层）。而图书馆管理员已经帮你把不该看的书页

粘起来了（规则层过滤掉 IGNORE 字段）。

Agent 面对 50 个工具时，它的上下文里只多了一张 50 行的表格。而不是 50 份 schema 文档。

热更新：让系统自己长出新能力

上面说了 CLI 接管工具管理，但这里有一个更深的设计——**工具列表不是静态的，它是每次激活时实时同步的。**

discover：不是"加载工具"，是"同步世界状态"

每次 Skill 被激活，第一件事是执行 `discover`：

```
bash pangu-cli.sh discover
```

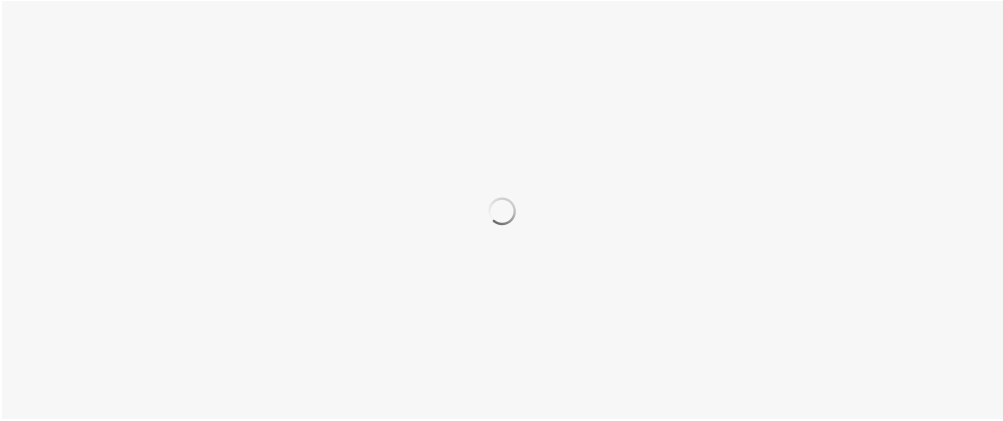
这个命令的 stdout **只返回一个变更摘要 JSON**：

```
{"status":"ok","total":5,"added_count":2,"removed_count":0,"added":["tool_a","tool_b"]
```

但它在内部悄悄做了一大堆事：

1. 通过 MCP 协议调用后端 `tools/list`，拿到最新工具列表
2. 与本地缓存做增量对比（新增了哪些、移除了哪些）
3. 为每个新工具生成独立的 `.meta` 参数文件
4. **自动从参数描述中提取枚举值**，写入 `.rules` 文件的 ENUM 行
5. **自动识别系统字段**（如 `id`、`update_time`），生成 IGNORE 规则
6. 扫描本地 `workflows/` 目录，更新工作流索引
7. 重新生成 `tools-index.md`

Agent 唯一感知到的，就是"增加了 2 个工具"。 它连新工具有几个参数都不知道——等用户真的要用时，才通过 `--meta-json` 按需获取。



这件事的精妙在于：不仅同步，还自动生成"使用规则"

普通的工具同步就是把后端返回的 JSON 存下来。但这套 CLI 做了一步额外的事——**它在同步的同时，自动为工具生成了人类需要手写的规则**：

```
# create_project 的自动生成 .rules 文件
IGNORE:id                    ← 系统自动分配，别让 Agent 传
IGNORE:insert_time           ← 时间戳，后端自动写
IGNORE:update_time
IGNORE:update_admin
NOTE:host 必须是已加白的有效域名
ENUM:auth_mode:0-预鉴权,1-SDK接入,2-openapi接入,3-未接入  #auto
```

这意味着：后端加了一个新工具，CLI 自动帮它识别出"哪些字段是系统字段不该暴露给 Agent"、"哪些字段有枚举值应该做成选项卡片"。**连规则都是自动生成的，而不是人写的。**

为什么这是为后续做铺垫

这套"自动发现 + 自动生成规则 + 按需加载"的机制，解决的核心问题是：**让 Skill 的能力可以无限扩展，而 Agent 的认知负担始终恒定。**

不管后端有 5 个工具还是 500 个，Agent 的上下文占用是一样的——一份索引表格。这为接下来要讲的 Workflow 机制打下了基础：因为 Workflow 内部会调用多个工具串联执行，如果工具管理本身是混乱的，Workflow 就无法可靠运行。

Workflow：真正解决"有工具，但解决不了实际问题"的设计

前面三节解决了"单个工具如何可靠运行"。但用户的真实需求从来不是"帮我调一个 API"——而是"帮我搭一套权限体系"。后者需要按正确顺序调用多个 API，中间还要跟用户交互确认，并且不能丢状态。

这就是 **Workflow（工作流）** 要解决的核心问题：**把散落的工具串成一条可靠的流水线。**

在展开它的内部设计之前，我先把 Workflow 最大的两个亮点拍在最前面——因为这两点决定了它不只是"一个编排引擎"，而是这套 Skill 真正的扩展性引擎。

第一个亮点：Workflow 是"长出来"的，不是"写出来"的

Workflow 不写在 Skill 代码里，它就是文件系统上的一组 Markdown 文件。

新增一个工作流的全部成本：

- 在 `workflows/` 目录下**新建一个文件夹** ← 完
- 不需要改 SKILL.md

- 不需要改 CLI 脚本
- 不需要重启任何东西，下次 discover 时 CLI 自动扫进索引

这意味着：**Skill 的业务能力可以无限横向扩展，而 Skill 本身的代码完全不动。** 后端今天新增一个业务场景？丢一个 workflow 文件夹进去。明天又一个？再丢一个。永远不需要发版改代码。

更妙的是——**Workflow 的 Markdown 是人也能手搓的**。一个 `WorkFlow.md` 加几个按文件名前缀排序的步骤文件，每个步骤文件就是一份 YAML front matter（声明 gate schema 和 automation）加一段 Markdown 正文（给执行时 Agent 看的指引）。

```
workflows/<新流程名>/
├── WorkFlow.md            ← name + description
└── references/
    ├── 01-xxx.md
    ├── 02-xxx.md
    └── ...
```

任何懂业务的人——不一定是开发——抄一份已有 Workflow 改改字段名和 instructions 就能定义出一个新流程。**扩展 Skill 能力的门槛，从"会写代码"降到了"会复制粘贴改 YAML"。**

> 当然手搓总要懂格式、要心细。所以我后面做了第二个 Skill `workflow-creator`，让 Agent 帮你把这件事也工程化（后文会讲）。但**手搓这条路始终是开放的**——它没有被工具链锁死。CLI 也好、人也好、另一个 Agent 也好，只要产出的文件结构对，Workflow 引擎都认。

这种"**扩展能力 ≠ 修改代码**"的设计，是这套系统最容易被忽视、但生命力最强的一点。

第二个亮点：Workflow 与 Skill 同构，可以无缝互转

如果你仔细看，Workflow 的目录结构和一个 Skill 的目录结构几乎是镜像的：

Skill	Workflow
`SKILL.md` （描述 + 入口）	`WorkFlow.md` （描述 + 入口）
`references/` 下的步骤文档	`references/` 下的步骤文档
通过 CLI 触发	通过 CLI 触发
在 discover 时被扫描	在 discover 时被扫描

这种同构不是巧合——是有意为之的。它带来一个非常顺手的工程红利：

一个写得好的 Skill，几乎可以原封不动地"塞进"另一个 Skill 当作 Workflow 复用。

举个具体的例子：你有一个独立的 `permission-set-skill` 处理权限配置；今天产品要做一个更大的"应用接入向导"，需要在它的中段嵌入一整套权限配置流程。怎么办？

- 不需要把 `permission-set-skill` 重写一遍
- 不需要把它的逻辑"塞进"新 Skill 的提示词
- **直接把它的 references 目录复制过来，改成 workflow 即可**

反过来也成立：一个 Workflow 跑得好，沉淀成熟之后可以独立成一个 Skill，直接发给其他团队用。

但 Workflow **不只是缩小版的 Skill**——它有 Skill 不具备的几样关键能力：

- **状态持久化**：跑到一半中断，第二天接着跑（详见 4.6）
- **强制门禁**：每步必须填齐 schema 才能推进（详见 4.5）
- **数据流自动接力**：上一步的输出自动喂给下一步的输入，不依赖 Agent 记忆（详见 4.7）
- **多步连锁自动执行**：Agent 提交一次数据，CLI 可能跑完后续 3 步才回来（详见 4.8）

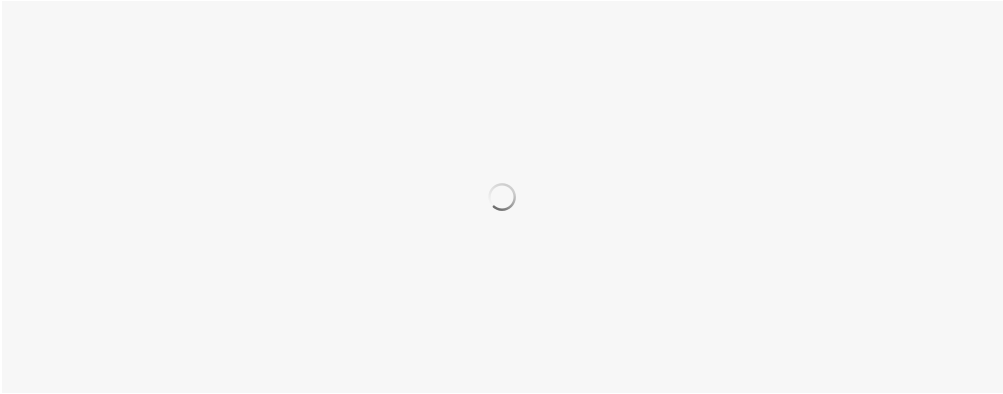
这些能力让 Workflow 在"多步骤、有状态、跨会话"的复杂业务场景里碾压裸 Skill。下面就一个个展开。

用一句话总结 Workflow 在做什么

如果非要用一个比喻：单个工具是食材，Workflow 就是食谱。

你有鸡蛋、面粉、糖、黄油——这些是单个工具。但只有食材不代表你能做出提拉米苏。你还需要一份食谱：告诉你先做什么后做什么、每步需要什么材料、做完长什么样算合格。

没有 Workflow 时，用户说"帮我配一套权限体系"，Agent 面对的是一堆散落的积木：`create_project`、`create_role`、`create_permission`……它得自己想该按什么顺序拼。这就像让一个从没拼过乐高的小孩面对 500 块零件——可能拼出来，也可能拼成四不像。更惨的是，他每次拼的路径还不一样。



有了 Workflow：

```
用户说"创建权限集"
        ↓ 自动匹配工作流
```

```
[步骤1: 收集项目信息] → [步骤2: 调 API 创建项目]
         ↓ 自动串联
[步骤3: 收集角色信息] → [步骤4: 调 API 创建角色]
         ↓
[步骤5: 展示摘要，确认完成]
```

Agent 不需要"理解"这五步之间的关系——它只需要走流水线：当前步给它什么指引，它就做什么事。"该怎么拼"这个知识，已经被固化在文件系统的目录结构里了。

discover 时 CLI 同时扫描工具和工作流，生成的索引里工作流排在工具前面：

```
# 可用工作流
| 工作流 | 功能 |
| `create-permission-set` | 创建完整的权限集配置 |

# 可用工具
| 工具 | 功能 |
| `create_project` | 创建新项目 |
```

当用户表达一个意图时，Agent 的匹配逻辑是：**先看有没有匹配的工作流，没有才降级到单个工具。** 高层意图能直接映射到一整套编排好的流程——而不是让 Agent 自己去拼凑。

Workflow 要解决的三个核心难题

在展开设计细节之前，先说清楚 Workflow 到底要应对什么：

难题	不用 Workflow 时的后果
流程顺序：5 步该按什么顺序走？	Agent 跳步骤、乱序执行、遗忘步骤
步间通信：第 2 步要用第 1 步的结果怎么办？	Agent 记不住 / 搞混 / 拼错字段名
中断恢复：做到一半 IDE 崩了怎么办？	一切从头再来，用户崩溃

这三个难题，分别对应了 Workflow 内部的三个设计机制。接下来逐一展开。

设计细节一：步进式披露——Agent 永远只看到当前一步

这是 Workflow 最核心的机制。

问题：把 5 个步骤一次性写进提示词时，AI 做完前两步就开始"即兴创作"。原因很简单——如果你给助理一张 5 条指令的清单，他做完第 2 条后瞄了第 4 条觉得更有意思，就跳过去了。

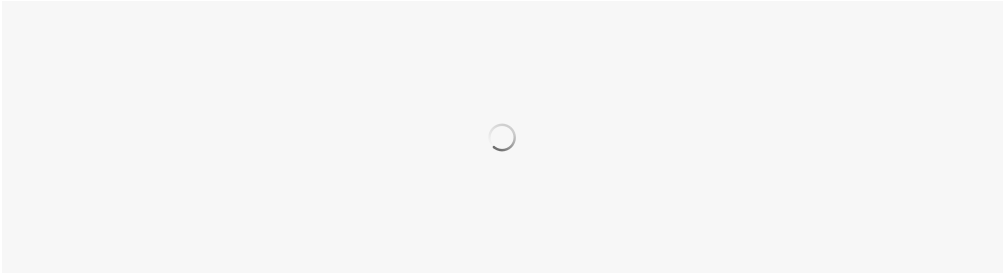
设计：流程不再是提示词里的一段文字，而是**文件系统里的一组步骤文件**：

```
workflows/create-permission-set/
├── WorkFlow.md            ← 只有名字 + 一句话描述
└── references/
    ├── 01-collect-project-info.md
    ├── 02-create-project.md
    ├── 03-collect-role-info.md
    ├── 04-create-role.md
    └── 05-summary.md
```

Agent 通过 CLI 与流程交互，只有两个命令：

- `--start`：启动流程，CLI 返回**第 1 步的内容**
- `--advance`：提交当前步数据，CLI 返回**下一步要做什么**

Agent 在任意时刻看到的，永远只有"当前这一步"。 前面的步骤已经过去（结果存档在 state 文件里），后面的步骤还没披露（Agent 压根不知道有什么）。



> AI 从来不需要"自由"。它需要的是"当前该做什么"的极度清晰。

Agent 不需要看到完整流程图，就像流水线上的工人不需要看到整个工厂的蓝图——他只要知道"现在面前这个零件该拧哪个螺丝"就够了。全局编排是工厂主管（CLI 状态机）的事。

而且——因为每一步是独立文件，改流程变成了文件操作：加一步 = 加一个文件，改顺序 = 改文件名前缀。不需要改任何代码。

设计细节二：Gate（门禁）——把开放题变成填空题

步骤拆开后，新问题来了：**Agent 不知道每一步到底该收集"几个"信息。**

比如"收集基础信息"这一步——Agent 可能漏问关键字段（下一步就报错），多问无关字段（浪费时间），或者把字段名拼错（CLI 无法对应）。

根因：**"完成"这个概念，对 Agent 来说是模糊的。** 什么叫"收集完了"，是它自己拍脑袋决定的。

设计：每个步骤文件的头部声明一个 **gate schema**——精确定义本步需要提交什么、什么类型、是否必填：

```
---
id:collect-project-info
type:interactive
gate:
schema:
    project_name:{type:string,required:true,desc:"项目名称"}
    host:        {type:string,required:true,desc:"服务域名"}
    admins:      {type:string,required:true,desc:"管理员 RTX"}
---
```

Agent 调 `--advance --gate-data '{...}'` 时，CLI 做三件事：

1. required 字段没给？**拒绝推进**，告诉 Agent 缺什么
2. schema 没声明的字段？**直接丢弃**
3. 全部满足？存储数据，推进下一步

这里有一个反直觉的点：约束越强，AI 反而越自由。

听起来矛盾，但逻辑很简单：当"完成标准"是确定性的 schema 时，Agent 的认知负担从"判断什么算完成"降级为"把这几个格子填好"。

类比生活：你让男朋友"给我准备个惊喜"，他不知道从哪下手，最后弄出一个你根本不想要的结果。但如果你说"明天帮我买一杯三倍厚抹少冰少少甜加量浓抹糯糯"——他 100% 能做对。Gate 的作用就是：**把开放题变成填空题。**

设计细节三：状态持久化——记忆不在 AI 脑子里

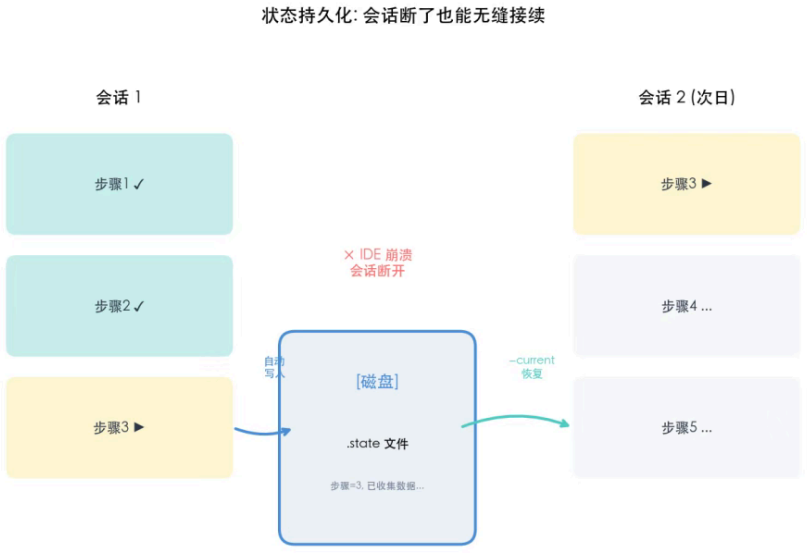
单次会话内流程稳定了。但真实使用中：IDE 崩了、晚上停了第二天接着做、切了个会话再回来——Agent 什么都不记得。

LLM 没有真正的记忆。上下文一断，前面所有信息蒸发。

设计：CLI 把整个流程的运行状态写到磁盘上的 JSON 文件（`.state`）：

- 当前走到第几步
- 每一步收集的 gate data
- 每一步执行的结果
- 整体状态：进行中 / 已完成 / 已中止

任何时候、任何会话、任何模型——调一个 `--current`，就能从断点续上。



Agent **不需要假装自己有记忆**了。它每次被激活只需要问 CLI"我们到哪了"，然后专心做当下这一步。就像接力赛——新跑手接过接力棒，不需要记住前面几棒跑了什么路线。

这个设计还有一个隐藏收益：**可审计**。每步的输入输出都记录在 `.state` 文件里——出了问题直接打开文件，就知道哪一步数据不对。

设计细节四：模板变量——步骤间的数据流由 CLI 管理

步骤独立了，但有依赖：第 2 步"创建项目"需要用第 1 步收集的 `project_name` 。

让 Agent 把数据带过去？又回到"让 AI 记东西"的老路。

设计：引入模板变量。在 automated 步骤的定义里直接引用前序步骤的字段：

```
---
id:create-project
type:automated
automation:
tool:create_project
input_mapping:
    name:"{{gate.collect-project-info.project_name}}"
    host:"{{gate.collect-project-info.host}}"
---
```

CLI 推进到这一步时，从 state 文件取值、渲染参数、执行工具——全程不经过 Agent。

两种引用语法：

- `{{gate.<step-id>.<field>}}` — 前序交互步骤的用户输入
- `{{result.<step-id>.<path>}}` — 前序自动步骤的 API 返回值

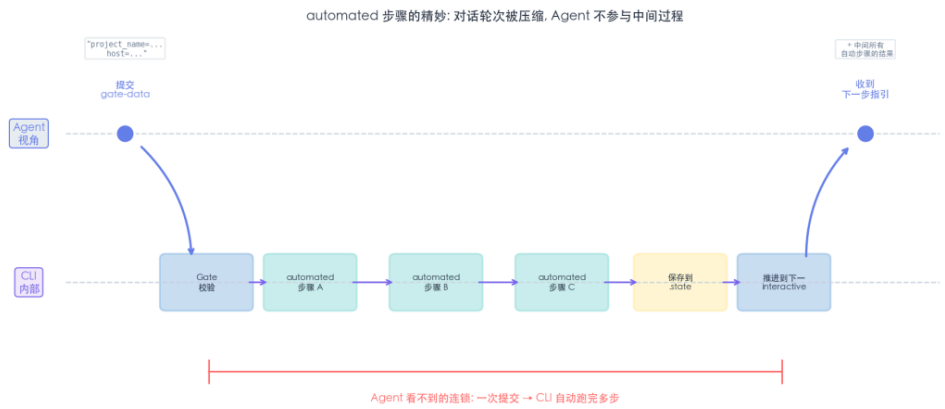
Agent 只关心当下的交互，数据的来龙去脉全由 CLI 管理。 就像你在网页上填表单——你不需要知道第 1 页填的手机号怎么传到第 3 页。那是后端的事。

设计细节五：三种步骤类型的协奏

Workflow 里的步骤分三种，每种承担不同角色：

类型	角色	谁干活
`interactive`	与用户交互（收集信息/确认）	Agent
`automated`	调用 API 完成操作	CLI 内部自动执行
`notification`	展示信息或阶段性总结	Agent 只做展示

精妙之处在于 `automated` 步骤。当 Agent 提交 gate data 后，CLI 内部这样运转：




```
Agent: --advance --gate-data '{"project_name":"权限中心","host":"perm.example.com"}'

CLI 内部（Agent 看不到）：
  1. 验证 gate data ✓
  2. 发现下一步是 automated
  3. 解析模板变量 → "权限中心"
  4. 调用工具：create_project --name "权限中心" --host "perm.example.com"
  5. 存储结果
  6. 下一步还是 automated → 继续执行
  7. 遇到 interactive → 停止，把控制权还给 Agent

Agent 收到：下一个 interactive 步骤的指引 + 所有中间 automated 步骤的执行结果
```

Agent 提交一次数据，CLI 可能连续做了三件事才把控制权还回来。 对话轮次被压缩——用户体验是"我提供了信息，系统一口气帮我搞定了好几步"，而不是被反复追问。

Workflow 是文件，不是代码

最后一个关键设计选择：Workflow 的定义是文件系统上的一组 Markdown 文件，不是代码。

```
workflows/create-permission-set/
├── WorkFlow.md                          ← YAML front matter: name + description
└── references/
    ├── 01-collect-project-info.md       ← interactive, gate: {project_name, host, adr
    ├── 02-create-project.md             ← automated, tool: create_project
    ├── 03-collect-role-info.md          ← interactive, gate: {role_name, role_type}
    ├── 04-create-role.md                ← automated, tool: create_role
    └── 05-summary.md                    ← interactive, 展示结果
```

收益：

- **业务人员也能改**——不用写代码，改 Markdown 就行
- **版本管理天然支持**——每步变更都有 git diff
- **复用极高**——把步骤文件复制到别的 Workflow 目录，改改模板变量就能跑
- **排序即逻辑**——文件名前缀决定顺序，重排步骤 = 重命名文件

自举：用 Skill 创造 Skill

到这里，运行时系统成型了。但我发现一个尴尬：**新建 Workflow 本身就是一个"多步骤、固定格式、确定性"的流程。**

手动创建 WorkFlow.md、步骤文件、YAML front matter、gate schema、automation 配置……容易出错，步骤繁琐。而这恰恰是这套系统最擅长处理的事情类型。

于是我做了第二个 Skill：workflow-creator

它的职责是把"用户描述业务流程 → 生成可执行的 Workflow 文件"也工程化：

- **Agent 做需求分析**：从用户描述里拆出步骤、确定类型、设计 gate schema、规划数据流
- **配套 CLI 做文件生成**：`init` 创建骨架，`add-step` 追加步骤，`validate` 校验完整性

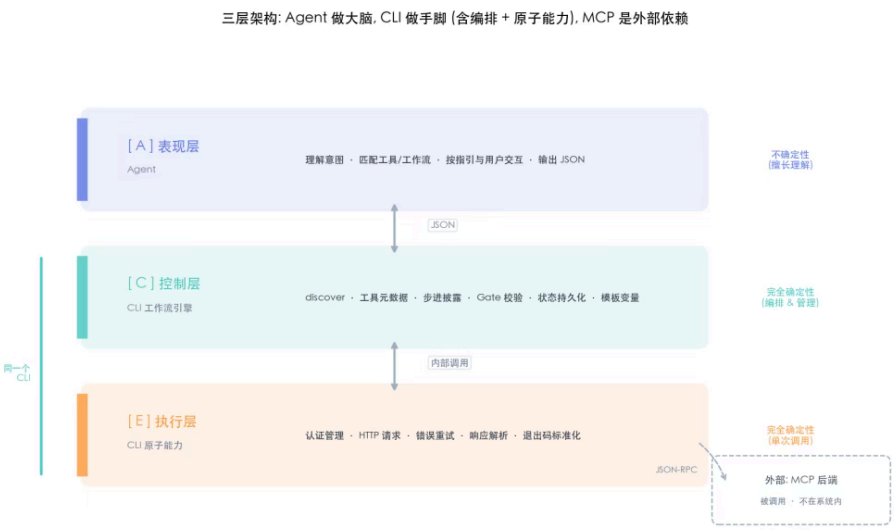
整个系统形成了**自洽的闭环**：

```
设计（Agent + workflow-creator）
    ↓
构建（CLI 写文件）
    ↓
执行（运行时 CLI 的工作流引擎）
    ↓
用户提出新需求 → 回到"设计"
```

这就是"**造工具的工具**"——系统不只能运行，还能自己长出新的能力。

全景图

把前面所有模块放在一起：



三层归属：

- **表现层** = Agent，负责理解和交互，保留不确定性但被限定在决策空间
- **控制层 + 执行层** = 同一个 CLI 的两个职责面。控制层是工作流引擎（discover / 步进 / Gate / 状态机 / 模板变量），执行层是原子 HTTP 调用（认证 / 重试 / 退出码）。两者在同一个 `pangu-cli.sh` 里，一起构成"CLI 这根支柱"
- **MCP 后端** = 外部被调用的服务，不在"系统内层"——CLI 通过 JSON-RPC 访问它

Agent 和 CLI 之间只用 JSON 通信；CLI 内部两层是函数级调用；CLI 到 MCP 后端走 JSON-RPC。整条链路上没有自然语言的歧义，没有"你理解一下"的灰色地带。

什么时候不需要这么做？

工程化是有成本的。以下场景，写个简单的 SKILL.md 就够：

场景	建议
一问一答的单次查询	不需要状态机和工作流
步骤 ≤ 2 且无恢复需求	直接写在提示词里
原型阶段跑通验证	先用提示词，复杂了再工程化
纯知识问答	根本不需要执行层

判断拐点：当你发现自己在提示词里反复写"注意不要……""确保先……""必须在……之前……"——你正在用自然语言模拟一个状态机。这就是该工程化的信号。

结语

回到开头的比喻。

LLM 是一条河，你没法改变它的本性——它就是概率性的、注意力有限的、没有持久记忆的。

但你可以给它修渠。渠道的走向是确定的，闸门的启闭是可控的，每一段蓄水量是可测的。河水在渠里流动时，它依然是那条河——灵动的、有理解力的、善于表达的。但它流向了你需要它去的地方。

这就是 Skill 工程化设计的全部哲学：**不是驯服 AI，而是为它构建一个它天然适合流淌的河道。**



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 8265

留言 7

写留言



Diao 广东 5月13日 6

谢谢，思路很新颖，很是受教，但是我觉得实际操作上还会存在很多挑战。比如当用户需求模糊或需要开放式探索时，严格的 Gate 会强行收集不必要的信息，导致交互僵化死板，没有了Agent在自然交互上的核心优势，越来越趋近于传统的编程。再则步骤间数据流应对无法预知的动态依赖还存在挑战。比如步骤3所需的 ID 是步骤2 API 返回结果的一个嵌套子字段。无论用 {{result.step2.id}} 还是 {{result.step2.data[0].id}}，都是一种硬编码的假设，一旦 API 返回结构变更，整个Workflow就会失败。



U.S.G 北京 5月13日 3

这个公众号蛮宝藏的🥰

首评



今麦郎 广东 5月13日 2

有没有具体的例子



拾雪掩碧空 广东 5月22日 1

您好，我是这篇文章的作者，有的，我发在腾讯内网km和我的博客有一篇后继文章专门讲具体例子，如果你需要的话可以私聊我



海 江苏 6月5日

回复 拾雪掩碧空：看了收获很大，我需要具体例子，谢谢



全险半挂大... 湖北 5月13日 1

"非常棒的工程实践分享，受益匪浅！🤭我在日常开发中也沉淀了类似的解决方案，开源了 git-workflow 工具（GitHub链接：https://github.com/spz-ecosystem/git-workflow），通过 Git Worktree 隔离、安全校验和流程封装，解决 AI 编程时代的多项目协同痛点。近期正好在探索 Agent 集群的开发和工程优化，看到你们分享的 Workflow.md 模式与状态管理、流程编排的实践经验，很有启发！马上尝试融合到我的工具链中，看看能否进一步解放开发者的协作效率。感谢分享！✨ "



hui 北京 5月13日

妙啊！